

Healthcare Data Quality Automation & Exception Reporting

How I Reduced a Six-Hour Data Review to 2.5 Hours

Project Overview

Role: Data Analyst

Engagement: July to October 2026

Primary Stakeholder: Healthcare Analyst

Data Volume: Approximately 250,000 records per batch

Tools: Python, SQL, Excel, Power Query and Power BI

The project started with a fairly straightforward problem: there was a lot of healthcare data to review, and reviewing it was taking too long.

Each file could contain around **250,000 records** across claims, patients, appointments, providers, billing, transactions, clinical records and insurance data.

Going through that amount of data manually was not a good use of an analyst's time.

More importantly, the analyst didn't really need to look at all 250,000 records.

They needed to find the records that were wrong or unusual.

That became the focus of my work.

The Problem I Was Trying to Solve

When I first looked at the process, the real problem wasn't the size of the file by itself.

It was this:

How do we find the small number of records that actually need attention without spending hours reviewing everything else?

There were several types of issues that could appear in the data:

- duplicate records;
- missing values;
- invalid dates;
- incorrect formats;
- invalid identifiers;
- values outside expected ranges;
- inconsistent codes;

- fields that were individually valid but did not make sense when compared with each other.

These were not all the same problem, so one generic "data quality check" was not enough.

I needed a repeatable way to test the data, identify what had failed and explain why it had failed.

Understanding the Data First

I didn't start by writing a large cleaning script.

I first needed to understand what was actually coming in.

The data came through **Excel, CSV and database/SQL sources**, and different healthcare areas were represented in the files.

I looked at things such as:

What does one record represent?

Which fields are expected to be populated?

What makes a record a duplicate?

What format should a particular value follow?

Which values have defined boundaries?

Which fields need to agree with one another?

That gave me the basis for defining the validation rules.

The Approach I Took

I broke the process into a simple flow:

Receive Data



Understand & Profile It



Prepare It for Validation



Run Data Quality Rules



Flag Records That Fail



Attach the Reason



Create Exception Output



Review Only What Needs Attention



Report the Results

The important change was near the end.

Instead of asking the Healthcare Analyst to review the whole file, I wanted the process to say:

"These are the records that need your attention, and this is why each one was flagged."

The Validation Checks

I organized the checks into eight areas.

1. Duplicate Detection

I checked for records that met the agreed duplicate definition.

The important part here was defining what a duplicate actually meant.

Two rows looking similar does not automatically make them duplicates.

The correct business fields had to be considered.

2. Missing Values

I checked fields where a value was expected but was null, blank or missing.

Again, I didn't treat every empty field as an error.

The rule had to apply to that field and record.

3. Date Validation

Dates were checked against the documented date rules.

This helped catch values that were missing, malformed or otherwise failed the expected validation.

4. Format Validation

Some values existed but were not in the format expected by the process.

These were separated from missing-value problems because they needed a different explanation.

5. Identifier Validation

Identifiers were checked against their documented rules.

This helped identify IDs that could not be considered valid under the agreed logic.

6. Range Validation

Where a field had documented boundaries, I checked whether values fell outside them.

7. Code Consistency

I checked values against the expected codes or mappings.

This was useful for identifying inconsistent classifications rather than just missing information.

8. Cross-Field Validation

This was one of the more useful checks.

Sometimes Field A is valid.

Field B is also valid.

But Field A + Field B together do not make sense.

Those problems will not be found by checking individual columns in isolation.

The project therefore included cross-field checks to identify relationships that failed the documented rules.

These eight validation categories are the ones covered by the actual project.

One Decision That Mattered: Record vs Failure

There was an important counting problem I had to handle.

Suppose one record fails three rules:

Record 10025

Missing Value ✕

Invalid Date ✕

Code Mismatch ✕

Is that:

three exceptions?

No.

It is **one exception record with three validation failures**.

That distinction matters when calculating the exception rate.

Otherwise, one bad record can be counted several times and make the data-quality problem look larger than it actually is.

So I kept two different levels:

RECORD LEVEL

250,000 records

↓

5,000 exception records

RULE LEVEL

5,000 exception records



Potentially more than 5,000 validation failures

The project therefore defined the exception rate using **distinct records**, while still retaining the individual failure reasons for analysis.

Making the Exceptions Useful

Finding a bad record wasn't enough.

Imagine receiving this:

Record 15782: FAILED

The first question would immediately be:

Why?

So failed records were given reason information.

For example:

Record	Result	Reason
R001	Pass	
R002	Exception	Missing required value
R003	Exception	Duplicate
R004	Exception	Invalid date
R005	Exception	Code mismatch
R006	Exception	Cross-field mismatch

This meant the analyst could move directly into investigation instead of first trying to work out what was wrong.

The project specifically maintained reason-coded exception output and source traceability for this purpose.

What the Data Told Us

After running the validation process, approximately **2% of processed records were identified as exceptions**.

For a batch of approximately 250,000 records, that meant a relatively small part of the overall population required attention.

The important thing about the 2% is that it was an **observed result**.

It was not a target.

It was not an acceptable error threshold.

It was simply what the validation found under the implemented rules.

That finding supported the main idea behind the project:

don't make an analyst spend the same amount of attention on every record when only a subset needs investigation.

Where Python and SQL Helped

With roughly 250,000 records in a batch, the checks needed to be repeatable.

Python and SQL supported the validation work rather than relying on row-by-row manual inspection.

The process was designed around:

Ingest



Validate



Apply Rules



Flag Exceptions



Assign Reasons



Generate Exception Output

The exact historical scripts and orchestration are not part of the retained project material, so I would not claim a specific production architecture in my portfolio. The confirmed workflow is the high-level validation process above.

Reconciliation Was Important Too

I didn't want the automation itself to become a black box.

If 250,000 records entered the process, I needed confidence that the outputs still reconciled to the batch.

That meant checking:

How many records were processed?

How many distinct records were exceptions?

How many individual validation failures occurred?

Do those numbers reconcile correctly?

This was particularly important because an exception record could have several failures.

Without separating those two levels, the Power BI numbers could easily be wrong.

Power BI Reporting

Once the validation output was controlled, Power BI was used to make the results easier to review.

For the portfolio version, I would organize the reporting into three views.

Data Quality Executive Overview

A quick view of:

- records processed;
- exception records;
- exception rate;
- validation coverage;
- review turnaround.

Exception Analysis

A more detailed view of:

- flagged records;
- validation reasons;
- applicable rule;

- exception investigation.

Validation Performance

A control-oriented view showing:

- validation coverage;
- record-level exceptions;
- rule-level failures;
- reconciliation.

This separation is deliberate.

The executive view answers:

"How healthy is this batch?"

The exception view answers:

"What went wrong?"

The validation view answers:

"Can I trust how these numbers were produced?"

The source report confirms Power BI reporting around these three portfolio-safe views and the need to prevent rule-level rows from multiplying record-level KPIs.

Testing the Process

Automation only saves time if people can trust it.

So validation itself also needed testing.

I checked whether:

- the input population reconciled;
- each rule produced the expected pass/fail result;
- failed records were correctly flagged;
- failure reasons were captured;
- a record with multiple failures was still counted as one exception record;
- the exception-rate calculation used the correct population;
- Power BI measures matched the validation output.

The project operated at the confirmed scale of approximately **250,000 records per batch**.

The Result

This is where the project had a clear measurable outcome.

Before the improved process, reviewing a file took approximately:

6 hours

After automated validation and targeted exception review:

2.5 hours

That is:

3.5 hours less per file

or approximately:

58.3% lower review turnaround

At the same time, the validation process surfaced an observed **2% record-level exception rate** across batches of approximately 250,000 records.

The value wasn't that Python could process a file.

The value was that the Healthcare Analyst could spend considerably less time looking for problems and more time reviewing the records that had actually been flagged.

What I Would Improve Next

If I were continuing the work, my next focus would be on making the validation framework easier to maintain.

I would keep a controlled rule catalogue containing:

Rule ID

Rule Name

Description

Applicable Field

Validation Logic

Failure Reason

Status

Version

Last Updated

That would make changes easier to test and explain.

I would also make sure every batch retained enough source and batch information to trace an exception back to its origin.

Whenever a validation rule changed, I would regression-test it before using the new logic on regular batches.

Finally, I would keep Power BI calculations tied closely to the governed validation output so reporting logic did not slowly drift away from the actual validation logic.

These are recommendations for extending the project, rather than claims about historical implementation.

What I Learned From This Project

The biggest lesson for me was that **data quality is not really about finding bad rows**.

The useful part is building a process where someone can answer:

What failed?

Why did it fail?

How many records are affected?

Can I trace them back to the source?

Are we counting the problem correctly?

Can the analyst focus only on what actually needs review?

The project also reinforced something I think is easy to overlook in reporting:

the grain of a metric matters.

One record can have multiple problems.

If that distinction isn't handled correctly, a technically correct dashboard can still tell the wrong business story.

My Contribution

I worked on this engagement as a **Data Analyst**, covering healthcare data review, validation-rule implementation, exception analysis, automation, reporting and quality assurance.

The work also involved requirements, business rules, testing and documentation.

I used:

Python | SQL | Excel | Power Query | Power BI

across the engagement.

Business Impact

Measure	Result
Data processed	~250,000 records per batch
Observed exception rate	2%
Review time before	~6 hours per file
Review time after	~2.5 hours per file
Time saved	~3.5 hours per file
Turnaround reduction	~58.3%

These are the supported project results. The 58.3% figure is specifically the reduction in **review turnaround**, not a claim of cost savings, productivity improvement, clinical improvement or compliance improvement.

Portfolio Summary

Problem

Reviewing high-volume healthcare files manually was taking approximately six hours per file.

What I Found

Only a subset of the data actually needed investigation, but the process needed a consistent way to identify those records.

What I Did

I helped implement repeatable validation covering duplicates, missing values, dates, formats, identifiers, ranges, code consistency and cross-field relationships.

Important Decision

I separated **exception records** from **individual validation failures** so the same record would not be counted multiple times in the exception rate.

How Review Improved

Failed records were separated and given reason information so analysts could focus directly on exceptions.

Reporting

Power BI provided visibility into processed volume, exceptions, validation results and review performance.

Result

Approximately **250,000 records processed per batch**

2% observed record-level exception rate

Review turnaround reduced from **6 hours to 2.5 hours per file**

Approximately **58.3% reduction in review turnaround**

Confidentiality

This public case study does not include patient information, PHI, real patient or provider identifiers, confidential client values, original source schemas, table names or column names.

If sample datasets, SQL scripts or Power BI screenshots are added to GitHub, they should use **synthetic data only**.

The underlying report also explicitly distinguishes the facts that are retained from details that should not be reconstructed as historical facts, including exact schemas, per-rule exception distributions, formal UAT sign-offs and production architecture.

Name : Chinmayee Surwade

Email : chinmyee.surwade@gmail.com